جزوه‌ی هفته‌ی ۴: الگوهای طراحی برای یادگیری ماشین

Strategy برای مدل‌ها، Observer برای کالبک‌های آموزش، Factory برای بارگذارهای داده و pytest

هوش مصنوعی برای مهندسی هوافضا و مکانیک (دکتری)، بخش صفر

> **منشأ محتوا:** محتوای جدید. جزوه‌های مدرس الگوهای طراحی نرم‌افزار را پوشش نمی‌دهند. مطالب الگوها با بازنویسی (نه رونویسی) از GeeksforGeeks (الگوی طراحی Strategy، الگوی طراحی Factory Method، Observer در پایتون، شروع کار با Pytest، Fixtureهای Pytest) و E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns* (1994) تهیه شده است؛ جزئیات پایتون از W3Schools (کلاس‌ها، وراثت، چندریختی، دیکشنری، Lambda). اصطلاحات فارسی با بوت‌کمپ پایتون کدآموز (kodamoz.com/fa/bootcamps/python) مقایسه شد. کمترین مربعات و k-NN برگرفته از C. M. Bishop, *Pattern Recognition and Machine Learning*, Ch. 3 و Sec. 2.5.2؛ توقف زودهنگام از Goodfellow, Bengio, Courville, *Deep Learning*, Sec. 7.8؛ و مدل جریان سوخت از J. D. Anderson, *Aircraft Performance and Design*, Ch. 5 است.

۱. مسئله

می‌خواهیم جریان سوخت $\dot{m}_f$ (گرم بر ساعت) یک پهپاد کوچک با موتور پیستونی را از سرعت V، ارتفاع h، نرخ صعود $\dot{h}$ و جرم m پیش‌بینی کنیم. هفته‌ی ۶ این را به‌عنوان مسئله‌ی رگرسیون بررسی می‌کند؛ این هفته نرم‌افزاری را می‌سازیم که هفته‌های ۵ تا ۱۳ از آن استفاده می‌کنند. تلاش اول معمولاً یک تابع است، `run_experiment(model_type, data_source, verbose, early_stop, patience)`، با یک زنجیره‌ی `if/elif` برای مدل، زنجیره‌ای دیگر برای منبع داده، و منطق گزارش‌دهی و توقف درون حلقه‌ی آموزش. این تابع از سه جهت شکست می‌خورد و هر سه در آزمایشگاه زنده نشان داده می‌شوند: نام غلط‌نوشته‌ی `"KNN"` به شاخه‌ی `else` می‌افتد و مدل پایه‌ی میانگین را **بدون هیچ خطایی** برمی‌گرداند؛ شاخه‌ی k-NN ویژگی‌ها را مقیاس‌بندی نمی‌کند و چون هیچ مقایسه‌ای در کار نیست کسی این خطا را نمی‌بیند؛ و هر مدل، منبع داده یا قابلیت آموزشی تازه همان یک تابع را ویرایش می‌کند، پس هیچ بخشی جداگانه آزمون‌پذیر نیست.

این تابع در سه **محور** مستقل تغییر می‌کند: مدل، آنچه حین آموزش رخ می‌دهد، و منبع داده. سه الگوی طراحی کلاسیک هر کدام یک محور را جدا می‌کنند:

آنچه تغییر می‌کند	الگو	کلاس‌های درس (`src/aerospace_ai/`)
مدل	Strategy (راهبرد) و Adapter (مبدل)	`models.Regressor`، `models.RegressionExperiment`
رفتار آموزش	Observer (ناظر)	`training.Trainer`، `training.Callback`
منبع داده	Factory (کارخانه)	`datasets.DatasetFactory`، `datasets.DatasetLoader`

هدف در هر سه **اصل باز/بسته** است: کد برای گسترش باز و برای ویرایش بسته باشد. افزودن مدل، کالبک یا بارگذار باید یعنی افزودن یک کلاس، نه ویرایش کدی که درست کار می‌کند.

مثال حل‌شده‌ی ۱: جریان سوخت در سرعت ۲۵ متر بر ثانیه

توان لازم برابر مجموع توان پسای مزاحم، پسای القایی و صعود است؛ توان محور برابر P_{req}/η_p و جریان سوخت برابر BSFC ضرب در توان محور:

$$P_{\text{req}} = \tfrac{1}{2}\rho V^3 S C_{D0} + \frac{2W^2}{\rho V S \pi e\, AR} + W\dot{h}, \qquad \dot{m}_f = \text{BSFC}\cdot\frac{P_{\text{req}}}{\eta_p}.$$

با $\rho = 1.225$ kg/m³، $V = 25$ m/s، $S = 1.5$ m²، $C_{D0} = 0.03$، $m = 30$ kg ($W = 294.20$ N)، $AR = 10$، $e = 0.8$، $P_{\text{par}} = 0.5\times1.225\times15\,625\times1.5\times0.03 = 430.66$ W؛ در پرواز تراز: BSFC $= 0.35$ kg/kWh و $\eta_p = 0.75$، $P_{\text{ind}} = 173\,106/1154.5 = 149.94$ W، توان محور $580.60/0.75 = 774.13$ W و $\dot{m}_f = 0.35\times0.77413 = 0.27095$ kg/h یعنی 270.95 گرم بر ساعت. صعود ۱ متر بر ثانیه $W\dot{h}/\eta_p = 392.3$ W اضافه می‌کند و سوخت به 408.24 گرم بر ساعت می‌رسد. مدل نسبت به V محدب است و حاصل‌ضرب $m\dot{h}$ را دارد، پس هیچ خط راستی دقیقاً بر آن منطبق نمی‌شود. مجموعه‌داده‌ی ۴۰۰ سطری درس (`fuel_flow_synthetic`) با بذر ۴ نویز اندازه‌گیری ۳ درصد دارد.

۲. Strategy: مدل به‌عنوان قطعه‌ای قابل تعویض

الگوی Strategy هر الگوریتم را در کلاس جداگانه‌ای پشت یک رابط مشترک قرار می‌دهد. **زمینه** (Context)، یعنی کدی که از الگوریتم استفاده می‌کند، ارجاعی به «یک راهبرد» نگه می‌دارد و فقط رابط را صدا می‌زند؛ هرگز نمی‌پرسد کدام الگوریتم را در دست دارد. تعویض یک انتساب است و الگوریتم تازه یک کلاس تازه.

سبک‌ترین راهبرد در پایتون یک تابع است: تابع‌ها شیء هستند، پس دیکشنری `{"mean": lambda ..., "median": lambda ...}` خودش یک جدول راهبرد است و کلید ناشناخته به‌جای عبور بی‌صدا خطای `KeyError` می‌دهد. تابع تا وقتی کافی است که راهبرد حالت و تنظیمات نداشته باشد. مدل‌ها هر دو را دارند (`coef_` آموخته‌شده، `k` یا `alpha` انتخاب‌شده) و هر بار به همان بررسی‌های ورودی نیاز دارند؛ از این رو درس از یک کلاس پایه‌ی انتزاعی استفاده می‌کند. کلاس `Regressor(ABC)` متدهای عمومی `fit` و `predict` دارد که شکل، متناهی بودن مقادیر و وضعیت برازش را یک بار بررسی می‌کنند و سپس قلاب‌های انتزاعی `_fit` و `_predict` را صدا می‌زنند؛ این‌ها متدهای قالبی‌اند، همان‌طور که `BaseSensor.add_reading` در هفته‌ی ۲ بود. راهبردهای مشخص عبارت‌اند از `MeanBaseline`، `LinearLeastSquares` (معادلات نرمال با جریمه‌ی اختیاری ریج)، `KNearestNeighbors` (با استانداردسازی اختیاری) و `GradientDescentRegressor`. زمینه‌ی `RegressionExperiment` یک راهبرد را روی مجموعه‌ی آموزش برازش می‌دهد، روی مجموعه‌ی آزمون ارزیابی می‌کند و یک `EvaluationResult` تغییرناپذیر برمی‌گرداند؛ متد `compare` فهرستی از راهبردها را روی یک تقسیم اجرا می‌کند.

مبدل (Adapter) یک رابط را به رابط دیگری تبدیل می‌کند. برآوردگرهای scikit-learn متدهای `fit` و `predict` دارند اما زیرکلاس `Regressor` نیستند؛ کلاس `SklearnRegressor(estimator)` هر شیئی را که *شکلش* با پروتکل `SupportsFitPredict` جور باشد می‌پوشاند. یک `typing.Protocol` همراه با `@runtime_checkable` متدهای شیء را بررسی می‌کند، نه تبار آن را: تایپ اردکی همراه با راهنمای نوع.

مثال حل‌شده‌ی ۲: سه راهبرد روی چهار پرواز

نقاط آموزش $(V, \dot{m}_f)$: $(18, 170)$، $(22, 200)$، $(26, 280)$، $(30, 400)$؛ نقاط آزمون $(25, 270)$ و $(29, 375)$.

- میانگین: $\bar{y} = 262.5$؛ خطاها -7.5 و -112.5؛ RMSE $= \sqrt{(56.25 + 12\,656.25)/2} = 79.73$ گرم بر ساعت.

- k-NN با $k = 2$ و بدون مقیاس‌بندی: همسایه‌های 25 نقاط 26 و 22 هستند و پیش‌بینی 240 می‌شود؛ همسایه‌های 29 نقاط 30 و 26 و پیش‌بینی 340. RMSE $= \sqrt{(900 + 1225)/2} = 32.60$ گرم بر ساعت.
- کمترین مربعات: $\bar{x} = 24$، $S_{xy} = \sum (x_i - \bar{x})(y_i - \bar{y}) = 1540$، $S_{xx} = 80$، شیب 19.25، عرض از مبدأ $= -199.5$ $- 19.25 \times 24 = -262.5$؛ پیش‌بینی‌ها 281.75 و 358.75؛ RMSE $= 14.18$ گرم بر ساعت.

درون بازه‌ی آموزش خط برنده است، اما در $V = 16$ m/s (مقدار واقعی حدود 162) خط 108.5 و k-NN مقدار 185 را پیش‌بینی می‌کند. بهترین راهبرد به پرسش بستگی دارد و دقیقاً به همین دلیل تعویض باید ارزان باشد.

روی کل داده با تقسیم زمانی ۳۰۰/۱۰۰، مدل‌های خطی (OLS، ریج و کاهش گرادیان) به RMSE آزمون 51.1 تا 51.5، k-NN مقیاس‌شده ($k = 5$) به 51.6، جنگل تصادفی از طریق مبدل به 37.65 و مدل پایه‌ی میانگین به 225.4 گرم بر ساعت می‌رسند. k-NN بدون مقیاس‌بندی 235.3 گرم بر ساعت می‌دهد، یعنی **بدتر از میانگین**: ارتفاع (۰ تا ۳۰۰۰ متر) در فاصله بر سرعت (۱۶ تا ۳۶ متر بر ثانیه) غلبه می‌کند. بین اجراها فقط `experiment.strategy` تغییر کرد و مقایسه خطای مقیاس‌بندی را آشکار کرد.

۳. Observer: رویدادهای آموزش بدون دست زدن به حلقه

حلقه‌ی آموزش ذی‌نفعان زیادی دارد: ثبت‌کننده‌ی تاریخچه، گزارشگر، قاعده‌ی توقف زودهنگام، نگهبان واگرایی، زمان‌بندی نرخ یادگیری و بعدها ذخیره‌کننده‌ی نقطه‌ی بازیابی. الگوی Observer به **سوژه** اجازه می‌دهد فهرستی از **ناظرها** نگه دارد و هنگام رخداد به همه‌ی آن‌ها اطلاع دهد، بی‌آنکه بداند آن‌ها چه می‌کنند. در کتابخانه‌ی درس سوژه `Trainer` است و ناظرها اشیای `Callback` با سه قلاب `on_train_begin`، `on_epoch_end` و `on_train_end`. قلاب‌ها عمداً انتزاعی نیستند، چون هر ناظر فقط به برخی رویدادها اهمیت می‌دهد. ناظرها فقط از طریق dataclass مشترک `TrainingState` (دوره، خطاها، نرخ یادگیری، `stop_training`، `stop_reason`) با هم ارتباط دارند و هرگز یکدیگر را صدا نمی‌زنند. `Trainer.fit` مدل را مقداردهی اولیه می‌کند، `on_train_begin` را اعلام می‌کند، سپس در هر دوره یک `train_step` برمی‌دارد، خطای اعتبارسنجی را حساب می‌کند، `on_epoch_end` را اعلام می‌کند و اگر کالبکی توقف خواسته باشد خارج می‌شود، و در پایان `on_train_end` را اعلام می‌کند. مدل فقط باید پروتکل `TrainableModel` را برآورده کند (`initialize`، `train_step`، `loss`، `get_state`، `set_state`)، پس همین حلقه در بلوک ۲ مدل‌های PyTorch را هم آموزش می‌دهد.

مثال حل‌شده‌ی ۳: یک گام کاهش گرادیان

برای $\hat{y} = w_0 + w_1 x$، داده‌ی $(1,2), (2,4), (3,5)$، شروع از $w = (0, 0)$ و تابع هزینه‌ی MSE: باقی‌مانده‌ها $(-2, -4, -5)$ و $L_0 = 15$ است. با سطرهای $A_i = (1, x_i)$ گرادیان برابر $\nabla L = \frac{2}{n} A^\top (Aw - y) = (-7.333, -16.667)$ است. یک گام با $\eta = 0.1$ نتیجه می‌دهد $w = (0.7333, 1.6667)$، پیش‌بینی‌های $(2.4, 4.0667, 5.7333)$ و $L_1 = 0.2341$. با $\eta = 0.5$ نتیجه $w = (3.667, 8.333)$ و $L_1 = 309$ است: گام از کمینه فراتر می‌رود. برای تابع هزینه‌ی درجه‌دو، کاهش گرادیان فقط وقتی پایدار است که $\eta < 2/\lambda_{\max}(H)$ با $H = \frac{2}{n} A^\top A$؛ اینجا $\lambda_{\max} = 11.09$ پس $\eta < 0.18$. روی داده‌ی استانداردشده‌ی سوخت $\lambda_{\max} = 2.17$ است و به همین دلیل $\eta = 0.1$ کار می‌کند و $\eta = 1.5$ واگرا می‌شود. کالبک `DivergenceGuard` اجرایی را که خطایش نامتناهی یا بیش از ده برابر خطای دوره‌ی اول شود متوقف می‌کند.

مثال حل‌شده‌ی ۴: توقف زودهنگام با دست

`EarlyStopping(patience=2, min_delta=0.05)` خطاهای اعتبارسنجی 1.00، 0.80، 0.78، 0.70، 0.69، 0.68 و 0.50 را می‌بیند. یک دوره فقط وقتی بهبود محسوب می‌شود که خطایش کمتر از best $- 0.05$ باشد. دوره‌های ۰، ۱ و ۳

بهبودند؛ دوره‌ی ۲ نه (0.78 کمتر از 0.75 نیست). شمارنده به ترتیب 0، 0، 1، 0، 1، 2 است، پس آموزش پس از دوره‌ی ۵ متوقف و وزن‌های دوره‌ی ۳ (0.70) بازگردانده می‌شود. با صبر ۳ اجرا به دوره‌ی ۶ و خطای بسیار بهتر 0.50 می‌رسید: صبر، هزینه‌ی محاسبه را با خطر رها کردن آموزش روی یک فلات مبادله می‌کند. روی داده‌ی سوخت، `min_delta = 1` به‌جای ۱۷۰ دوره پس از ۳۹ دوره متوقف می‌شود و RMSE آزمون همان است (51.19 در برابر 51.21 گرم بر ساعت).

دو قاعده‌ی عملی: ناظرها به **ترتیب اتصال** اجرا می‌شوند (گزارشگرِ پیش از `StepDecay` نرخ قدیم را چاپ می‌کند، هرچند آموزش یکسان است)، و هر ناظر باید شمارنده‌هایش را در `on_train_begin` صفر کند.

۴. Factory: داده را با نامش بخواه

منبع داده محور سوم است: امروز یک مولد مصنوعی و یک فایل CSV، هفته‌ی بعد لاگ نویزی LiDAR و در هفته‌ی ۱۱ پوشه‌ای از تصاویر دوربین. الگوی Factory ساخت شیء را از کد مشتری بیرون می‌برد: مشتری نام چیزی را که می‌خواهد می‌گوید و فقط به محصول انتزاعی `DatasetLoader` وابسته است که تنها متدش، `load()`، یک `Dataset` برمی‌گرداند (dataclass تغییرناپذیری با `X`، `y`، نام‌ها و متد `split` زمانی بدون نشت). کارخانه‌ی پایتونی یک **رجیستری** است: دیکشنری سطح کلاس `_registry: ClassVar[dict[str, type[DatasetLoader]]]` که با دکوراتور کلاس `@DatasetFactory.register("name")` پر می‌شود. دکوراتور فقط تابعی است که کلاس را دریافت، ذخیره و بدون تغییر برمی‌گرداند. `DatasetFactory.create(name, **kwargs)` بارگذار را با تنظیماتش می‌سازد و `load(name, **kwargs)` متد `load()` را هم صدا می‌زند. نام تکراری و کلاسی که بارگذار نیست رد می‌شوند و نام ناشناخته خطای `UnknownDatasetError` را همراه با نزدیک‌ترین نام‌های ثبت‌شده (به کمک `difflib`) می‌دهد. کتابخانه سه بارگذار دارد: `csv` (هر جدولی)، `flight_log_csv` (لاگ هفته‌ی ۱ با ستون مشتق‌شده‌ی ضریب بار، با استفاده‌ی دوباره از `FlightLog`) و `fuel_flow_csv`. منبع تازه یک کلاس دکوراتورخورده است؛ در آزمایشگاه `fuel_flow_synthetic` شش خط می‌شود.

۵. آزمون الگوها با pytest

الگوها آزمون نویسی را آسان‌تر می‌کنند چون هر قطعه فقط با یک رابط حرف می‌زند. سه روش در `tests/test_models.py`، `test_training.py` و `test_datasets.py` به کار رفته است (۹۶ آزمون در حدود ۲ ثانیه):

- **آزمون‌های قراردادی.** یک fixture با `params=` همه‌ی راهبردها را تولید می‌کند و همان شش آزمون روی هر کدام اجرا می‌شود: `fit` خود شیء را برمی‌گرداند، پیش‌بینی‌ها شکل درست دارند، `predict` پیش از `fit` خطای `NotFittedError` می‌دهد، تعداد ویژگی نادرست و داده‌ی بد خطای `ValueError` می‌دهند و هر مدل بهتر یا برابر با مدل پایه‌ی میانگین است. مدل تازه‌ای که به فهرست اضافه شود رایگان آزموده می‌شود. این همان اصل جایگزینی لیسکوف در قالب کد است: هر راهبرد باید هر جا که `Regressor` انتظار می‌رود کار کند.
- **بدل‌های آزمون.** `SpyCallback` رویدادهایی را که دریافت می‌کند ثبت می‌کند، پس یک assert ترتیب دقیق اطلاع‌رسانی را می‌سنجد. `ScriptedModel` فهرست ثابتی از خطاهای اعتبارسنجی برمی‌گرداند و مثال حل‌شده‌ی ۴ را به آزمون تبدیل می‌کند. `FakeEstimator` مبدل را بدون scikit-learn می‌آزماید.
- **جداسازی.** یک fixture بارگذاری موقتی ثبت و پس از `yield` حذف می‌کند؛ `tmp_path` برای فایل‌ها، `capsys` برای خروجی چاپی و `pytest.approx` برای اعداد اعشاری.